周报3

本周已刷题目

洛谷：普及 $\times 1$

力扣：简单 $\times 2$ 中等 $\times 3$ 困难 $\times 2$

牛客：中等 $\times 2$

- 排序
  1. P1177 【模板】排序 (**)
  2. 912. 排序数组 (随机快速排序) (**)
  3. 215. 数组中的第K个最大元素 (**)
  4. 912. 排序数组 (堆排序) (**)
- 归并分治
  1. 计算数组的小和*牛客题霸*牛客网 (**)
  2. 493. 翻转对 (***)
- 数组
  1. 1200. 最小绝对差 (*)
  2. 744. 寻找比目标字母大的最小字母 (*)
- 堆
  1. 23. 合并 K 个升序链表 (***)
  2. 线段重合*牛客题霸*牛客网 (**)

学习知识点

1. 归并分治

   > 1. 先看一个问题的答案是否等于左部分+右部分+跨越左右产生的答案；
   > 2. 计算跨越左右产生的答案时，若左右两侧有序，计算会不会更加便利；
   > 3. 若以上2点成立，则可以试着用归并分治的思想去解决。

2. 快速排序

   > 算法中主要部分含有随机的，计算复杂度要看数学期望
   >
   > 1. 随机选择数组中一个数作为基准，
   > 2. 划分三个区域，`==`基准元素的为基准区，对小于和大于区递归
   > 3. 划分3个区域的快速排序相较划分2个区域的，能更高效处理有相等元素的情况

3. 堆排序

   > 堆结构是完全二叉树，即每层的节点尽可能满。
   >
   > 构建大根堆，排序时依次把最大值换到数组末尾，对交换过来的堆顶元素重新调整为大根堆。

4. 堆

> `C++`里优先队列`priority_queue`就是堆结构，默认是大顶堆，要想实现小顶堆可以用lambda表达式写比较器来实现，返回为true表示第二个元素优先级高，应排在堆顶。需要头文件`queue`和`functional`。

刷题笔记

- 排序
  - P1177 【模板】排序 (**)

    > 迭代实现归并排序，时间复杂度$O(N*logN)$
    >
    > 1. 设置步长为1，每次迭代结束后翻倍；
    > 2. 内层循环通过步长把数组分开，依次把每组的左右数组归并；
    > 3. $m+1>=N-1$则说明这一组已经没有右组了，内层循环结束；

  - 912. 排序数组 (**)

    > 三路随机快速排序，时间复杂度$O(N*logN)$，空间复杂度$O(logN)$
    >
    > 1. 用随机选择数组内的`pivot`元素；
    > 2. 划分三个区域，`pvtl`是等于区域的左边界，`pvtr`是等于区域有边界；
    > 3. 递归小于和大于区域；

  - 215. 数组中的第K个最大元素 (**)

    > 随机快速选择，根据快速排序改版得到
    >
    > - 时间复杂度: `O(N)`
    > - 空间复杂度: `O(1)`
    >
    > 1. 找第k大的数相当于找升序排序后(n - k)位置的数；
    > 2. 随机选基准元素`pivot`，划分为`<pivot`，`==pivot`，`>pivot3`个区域，检查(n - k)是否是基准元素，是则直接返回；
    > 3. 对左右区域再找基准元素划分区域；

  - 912. 排序数组 (堆排序) (**)

    > - 时间复杂度$O(N*logN)$
    > - 空间复杂度$O(1)$
    >
    > 构建大根堆，排序时依次把最大值换到数组末尾，对交换过来的堆顶元素重新调整为大根堆
    >
    > 从底建堆的过程时间复杂度更低，为$O(N*logN)$，排序时间复杂度一样。
    >
    > 1. 堆元素下沉，先检查左右子节点是否越界，再比较出最大的子节点，若当前元素比最大子节点小，则交换；
    > 2. 排序，堆顶元素即为最大元素，交换到数组末尾，让`size`减少，`heapify`下沉调整时不会访问已经排序的末尾元素；

- 归并分治
  - 计算数组的小和*牛客题霸*牛客网 (**)

    > 用归并分治解决，时间复杂度$O(N*logN)$
    >
    > 1. 分治：把问题变成求左部分答案+右部分答案+合并两部分产生答案；

    2. 求合并两部分产生的答案，假设左右两部分都有序，用双指针滑窗，`sum`统计左部<=右部`j`索引元素的和；
        - 判断左部`i`元素是否<=右部`j`元素，若是，则`sum + i`，`i`指针右移；
        - 因为左右两部分有序，右部分`j`在左部分的小和`sum`也包含于`j + 1`在左部分的小和，所以`i`指针不用回退，
        - 用`i`的左侧元素和`sum`加上满足<=`j + 1`的左部分元素，并移动`i`指针，
        - 得到新的`sum`就是当前`j`在左部的小和；
        - 把每个`sum`加起来得到的就是合并两部分产生的答案；
    3. 归并这两部分，使之有序；

- 493. 翻转对 (***)

  归并分治，时间复杂度$O(N*logN)$

  1. 分治，递归回溯条件为`l == r`，即数组长度为1，天然有序；
  2. 用双指针滑窗统计逆序对，数据类型强制转换为`long long`，防溢出；
  3. 归并，把2个有序数组合并为一个；

- 堆
  - 23. 合并 K 个升序链表 (***)

    快速找到所有链表当前的最小节点，用小根堆实现

    时间复杂度:$O(N*logk)$，k为数组长度，即链表个数

    空间复杂度: $O(k)$

    1. 将每个非空链表的头节点入堆；
    2. 循环拼接，弹出堆顶最小节点，拼接到结果链表；若该节点有后继，将后继入堆；
    3. 终止条件，堆为空时，所有节点拼接完成，返回结果链表头；

  - 线段重合*牛客题霸*牛客网 (**)

    1. 先按开始区间排序；
    2. 用小根堆处理每个区间，弹出结束边界在左边界前的区间；
    3. 计算最大堆内元素

下周计划

学习堆结构相关问题

代码

1.排序

1. P1177 【模板】排序

   迭代实现归并排序

```cpp
#include <iostream>
#include <array>
using namespace std;
enum {MAXN = 100000};
```

```cpp

void mergeSort(array<int,MAXN> &arr, array<int,MAXN> &assistant, int n);
void merge(array<int,MAXN> &arr, array<int,MAXN> &assistant, int l, int m, int r);

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    array<int,MAXN> arr {};
    array<int,MAXN> assistant {};
    int n;

    cin >> n;
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    mergeSort(arr, assistant, n);

    for (int i = 0; i < n; i++) {
        cout << arr[i] << ' ';
    }
}

void mergeSort(array<int,MAXN> &arr, array<int,MAXN> &assistant, int n) {
    int step = 1;
    int l, m, r;
    while (step < n) {
        // 步长>=数组长度则结束
        l = 0;
        while (l < n) {
            // 分组，对每组依次归并
            m = l + step - 1;
            if (m + 1 >= n) {
                // 没有右侧数组了
                break;
            }
            // 维护r
            r = l + (step << 1) - 1 < n ? l + (step << 1) - 1 : n - 1;
            merge(arr, assistant, l, m, r);
            l = r + 1; // l到达下一组
        }
        step <<= 1; // 左移一位，等同于*2
    }
}

void merge(array<int,MAXN> &arr, array<int,MAXN> &assistant, int l, int m, int r)
{
    int i = l;
    int a = l;
    int b = m + 1;
    while (a <= m && b <= r) {
        assistant[i++] = arr[a] <= arr[b] ? arr[a++] : arr[b++];
```

```
    }
    while (a <= m) {
        assistant[i++] = arr[a++];
    }
    while (b <= r) {
        assistant[i++] = arr[b++];
    }
    for (i = l; i <= r; i++) {
        arr[i] = assistant[i];
    }
}
```

2. 912. 排序数组 (**)

> 随机快速排序

```
class Solution {
public:
    vector<int> sortArray(vector<int>& nums) {
        int n = nums.size();
        if (n <= 1) {
            return nums;
        }
        srand(time(nullptr));
        quickSort(nums, 0, n - 1);
        return nums;
    }

    void quickSort(vector<int>& nums, int l, int r) {
        if (l >= r) {
            return;
        }
        // 随机，让时间复杂度期望变为O(N * log N)
        int pivot = nums[rand() % (r - l + 1) + l];
        // 确定主轴元素位置
        int pvtl, pvtr;
        partition(nums, l, r, pivot, pvtl, pvtr);
        quickSort(nums, l, pvtl - 1);
        quickSort(nums, pvtr + 1, r);
    }

    // 优化版partition，三路快排
    void partition(vector<int>& nums, int l, int r, int pivot, int& pvtl, int&
pvtr) {
        pvtl = l, pvtr = r;
        int i = l;
        while (i <= pvtr) {
            if (nums[i] < pivot) {
                // 小于主轴元素的数交换到<区右边
                swap(nums, pvtl, i);
                pvtl++, i++;
            } else if (nums[i] == pivot) {
```

```
                // ==区本来就在主轴
                i++;
            } else {
                // 大于主轴元素的交换到>区
                swap(nums, i, pvtr);
                pvtr--;
            }
        }
    }

    void swap(vector<int>& nums, int i, int j) {
        int temp = nums[i];
        nums[i] = nums[j];
        nums[j] = temp;
    }
};
```

3. 215. 数组中的第K个最大元素 (**)

```
class Solution {
public:
    int pvta, pvtb; // 基准元素区的左右边界
    int findKthLargest(vector<int>& nums, int k) {
        srand(time(nullptr));
        // 找第k大的数相当于找升序排序后(n - k)位置的数
        return randomSelect(nums, nums.size() - k);
    }

    // 升序排序后k位置的数
    int randomSelect(vector<int>& nums, int k) {
        int l = 0;
        int r = nums.size() - 1;
        int pivot;
        while (l <= r) {
            // 随机选基准元素
            pivot = nums[rand() % (r + 1 - l) + l];
            partition(nums, l, r, pivot);
            if (k < pvta) {
                r = pvta - 1;
            } else if (k > pvtb) {
                l = pvtb + 1;
            } else {
                return nums[k];
            }
        }
        return 0;
    }

    // 三路分区，分区后pvta和pvtb就是基准元素左右边界
    void partition(vector<int>& nums, int l, int r, int pivot) {
        pvta = l, pvtb = r;
        int i = l;
        while (i <= pvtb) {
```

```
            if (nums[i] == pivot) {
                // 基准区
                i++;
            } else if (nums[i] < pivot) {
                // 小于区域扩大
                swap(nums, pvta, i);
                pvta++, i++;
            } else {
                // 大于区域扩大
                swap(nums, i, pvtb);
                pvtb--;
            }
        }
    }

    void swap(vector<int>& nums, int i, int j) {
        int temp = nums[i];
        nums[i] = nums[j];
        nums[j] = temp;
    }
};
```

4. 912. 排序数组 (堆排序) (**)

```
class Solution {
public:
    vector<int> sortArray(vector<int>& nums) {
        int n = nums.size();
        if (n <= 1) {
            return nums;
        }
        bottom_heapSort(nums);
        // top_heapSort(nums);
        return nums;
    }

    // 向上插入元素
    void heapInsert(vector<int>& nums, int i) {
        while (nums[i] > nums[(i - 1) / 2]) {
            swap(nums, i, (i - 1) / 2);
            i = (i - 1) / 2;
        }
    }

    // 下沉
    void heapify(vector<int>& nums, int i, int size) {
        // 左孩子
        int l = 2 * i + 1;
        while (l < size) {
            // 检查不越界，才有子节点
            int best = l + 1 < size && nums[l + 1] >= nums[l] ? l + 1 : l;
            best = nums[best] > nums[i] ? best : i;
            if (best == i) {
```

```
                break;
            }
            swap(nums, best, i);
            i = best;
            l = 2 * i + 1;
        }
    }

    // 从底建立大根堆，并弹出最大值并排序
    void bottom_heapSort(vector<int>& nums) {
        int n = nums.size();
        for (int i = n - 1; i >= 0; i--) {
            heapify(nums, i, n);
        }
        int size = n;
        while (size > 1) {
            // 交换最大值到数组末尾
            swap(nums, 0, size - 1);
            size--; // 弹出最大值
            heapify(nums, 0, size); // 下沉，调整
        }
    }

    // 从顶建立大根堆，并弹出最大值并排序
    void top_heapSort(vector<int>& nums) {
        int n = nums.size();
        for (int i = 0; i < n; i++) {
            heapInsert(nums, i);
        }
        int size = n;
        while (size > 1) {
            // 交换最大值到数组末尾
            swap(nums, 0, size - 1);
            size--;
            heapify(nums, 0, size);
        }
    }

    void swap(vector<int>& nums, int i, int j) {
        int temp = nums[i];
        nums[i] = nums[j];
        nums[j] = temp;
    }
};
```

2.归并分治

1. 计算数组的小和*牛客题霸*牛客网 (**)

```
#include <iostream>
#include <array>
```

```cpp
using namespace std;
enum{MAXN = 100001};

int n;
array<int, MAXN> arr;
array<int, MAXN> assistant;

long long smallSum(int l, int r);
long long merge(int l, int m, int r);

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    while (cin >> n) {
        for (int i = 0; i < n; i++) {
            cin >> arr[i];
        }
        cout << smallSum(0, n - 1) << '\n';
    }
    return 0;
}

long long smallSum(int l, int r) {
    if (l >= r) {
        return 0;
    }
    int m = (l + r) / 2;
    return smallSum(l ,m) + smallSum(m + 1, r) + merge(l, m, r);
}

long long merge(int l, int m, int r) {
    long long result = 0;
    // 统计部分
    for (int i = l, j = m + 1, sum = 0; j <= r; j++) {
        // 对每个j位置求小和sum
        while (i <= m && arr[i] <= arr[j]) {
            // i不越界且i位置元素满足<=j位置元素
            sum += arr[i++];
        }
        result += sum;
    }

    // merge部分
    int i = l, a = l, b = m + 1;
    while (a <= m && b <= r) {
        assistant[i++] = arr[a] <= arr[b] ? arr[a++] : arr[b++];
    }
    while (a <= m) {
        assistant[i++] = arr[a++];
    }
    while (b <= r) {
        assistant[i++] = arr[b++];
```

```
    }
    // 拷贝
    for (i = 0; i <= r; i++) {
        arr[i] = assistant[i];
    }
    return result;
}
// 64 位输出请用 printf("%lld")
```

2. 493. 翻转对 (***)

```
class Solution {
public:
    enum{MAXN = 50001};
    array<int, MAXN> assistant; // 固定长度数组，减小开销

    int reversePairs(vector<int>& nums) {
        int n = nums.size();
        return recursion(nums, 0, n - 1);
    }

    int recursion(vector<int>& nums, int l, int r) {
        if (l == r) {
            return 0;
        }
        // 分治
        int m = (l + r) / 2;
        return recursion(nums, l, m) + recursion(nums, m + 1, r) + merge(nums, l, m, r);
    }

    int merge(vector<int>& nums, int l, int m, int r) {
        // 统计逆序对
        int res = 0;
        int a, b, pairs;
        for (a = l, b = m + 1, pairs = 0; a <= m; a++) {
            while (b <= r && (long long)nums[a] > ((long long)nums[b] << 1)) {
                pairs++;
                b++;
            }
            res += pairs;
        }

        // 归并
        int i = l;
        a = l, b = m + 1;
        while (a <= m && b <= r){
            assistant[i++] = nums[a] <= nums[b] ? nums[a++] : nums[b++];
        }
        while (a <= m) {
            assistant[i++] = nums[a++];
```

```
        }
        while (b <= r) {
            assistant[i++] = nums[b++];
        }

        for (i = l; i <= r; i++) {
            nums[i] = assistant[i];
        }
        return res;
    }
};
```

3.数组

1. 1200. 最小绝对差

```
class Solution {
public:
    vector<vector<int>> minimumAbsDifference(vector<int>& arr) {
        vector<vector<int>> result;
        sort(arr.begin(), arr.end());
        int diff = INT_MAX;
        int n = arr.size();
        // 找最小绝对差
        for (int i = 0; i < n - 1; i++) {
            if (arr[i + 1] - arr[i] < diff) {
                diff = arr[i + 1] - arr[i];
            }
        }

        for (int i = 0; i < n - 1; i++) {
            if (arr[i + 1] - arr[i] == diff) {
                result.push_back({arr[i], arr[i + 1]});
            }
        }
        return result;
    }
};
```

2. 744. 寻找比目标字母大的最小字母 (*)

```
class Solution {
public:
    char nextGreatestLetter(vector<char>& letters, char target) {
        int l = 0;
        int r = letters.size() - 1;
        if (letters[r] <= target) {
            return letters[0];
        }
        int res = 0;
```

```
        while (l <= r) {
            int m = l + (r - l) / 2;
            if (letters[m] <= target) {
                l = m + 1;
            } else {
                res = m;
                r = m - 1;
            }
        }
        return letters[res];
    }
};
```

4.堆

1. 23. 合并 K 个升序链表 (***)

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
 * };
 */
class Solution {
public:
    ListNode* mergeKLists(vector<ListNode*>& lists) {
        // 小顶堆,内部元素为ListNode*，底层为vector，比较器定义lambda表达式实现小顶堆
        priority_queue<ListNode*, vector<ListNode*>, function<bool(const ListNode*, const ListNode*)>> heap([](const ListNode* a, const ListNode* b) {
            return a->val > b->val;
        });
        for (int i = 0; i < lists.size(); i++) {
            if (lists[i] != nullptr) {
                heap.push(lists[i]);
            }
        }
        if (heap.empty()) {
            return nullptr;
        }

        // 先弹出一个节点，做总头部
        ListNode* head = heap.top();
        heap.pop();
        ListNode* pre = head;
        if (head->next != nullptr) {
            heap.push(head->next);
        }
```

```
        // 循环取堆顶最小节点，拼接成新链表
        while (!heap.empty()) {
            ListNode* cur = heap.top();
            heap.pop();
            pre->next = cur; // 拼接当前节点
            pre = cur; // 指针后移
            if (cur->next != nullptr) {
                heap.push(cur->next);
            }
        }
        return head;
    }
};
```

2. 线段重合 *牛客题霸*牛客网 (**)

```
#include <iostream>
#include <array>
#include <algorithm>
#include <functional>

using namespace std;
enum {MAXN = 10001};
array<array<int, 2>, MAXN> intervals;
array<int, MAXN> heap;

void swap(int i, int j);
int count_lines(int n);
void add(int end, int& size);
void pop(int& size);

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;


    while (cin >> n) { // 注意 while 处理多个 case
        for (int i = 0; i < n; i++) {
            cin >> intervals[i][0] >> intervals[i][1];
        }

        cout << count_lines(n) << '\n';
    }
    return 0;
}

int count_lines(int n) {
    sort(intervals.begin(), intervals.begin() + n, [](const array<int, 2>& a,
    const array<int, 2>& b) {
        return a[0] < b[0];
    });

```

```
    int size = 0;
    int res = 0;
    for (int i = 0; i < n; i++) {
        while (size > 0 && heap[0] <= intervals[i][0]) {
            // 堆里区间尾小于新区间头
            pop(size);
        }
        add(intervals[i][1], size);
        res = size > res ? size : res;
    }
    return res;
}

void add(int end, int& size) {
    heap[size] = end;
    int i = size++;
    while (i > 0 && heap[i] < heap[(i - 1) / 2]) {
        swap(i, (i - 1) / 2);
        i = (i - 1) / 2;
    }
}

void pop(int& size) {
    swap(0, size - 1);
    size--;
    int i = 0;
    int l = 1;
    while (l < size) {
        // 找最小子节点
        int best = l + 1 < size && heap[l + 1] < heap[l] ? l + 1 : l;
        best = heap[best] < heap[i] ? best : i;
        if (best == i) {
            break; // 当前节点无更小子节点
        }
        swap(best, i);
        i = best;
        l = 2 * i + 1;
    }
}

void swap(int i, int j) {
    int temp = heap[i];
    heap[i] = heap[j];
    heap[j] = temp;
}
```